Format Penjelasan Materi — Ketentuan Penulisan Isi Materi

> **Dokumen ini adalah KETENTUAN untuk AI.** Dipakai saat menulis **satu file materi** yang menjadi isi dari satu poin roadmap.
> **Berlaku untuk:** materi **konsep/dasar pemrograman** (folder `.../Fundamental/<Teknologi>/...`).
> **Pasangan:** cara menyusun roadmap-nya ada di `Format Roadmap.md`.
> **Untuk materi project, gunakan `Format Penjelasan Materi Project.md`.**
> **Untuk bidang non-pemrograman (Bahasa, Filsafat, Psikologi, Sejarah, Seni), gunakan `Format Penjelasan Materi Non-Programming.md`.**

---

0. Tujuan & Kapan Dipakai

Setiap poin di roadmap (mis. `1. PHP adalah Server-Side – ...`) harus punya **satu file materi** yang menjelaskannya sampai paham.

File materi ditulis ketika:

- AI diminta "buatkan materi nomor N dari roadmap X", atau
- Melengkapi poin roadmap yang belum punya file.

Contoh lokasi & nama file:

```
Belajar/IT/Pemrograman/Website/Fundamental/PHP/
├── Level 1 – Fondasi/
│   └── A. Cara PHP Bekerja & Setup/
│       ├── 1. PHP adalah Server-Side – Cara Kerja yang Fundamental.md
│       ├── 2. Setup Environment – XAMPP atau Laragon.md
│       └── ...
```

Pola path: `Belajar/IT/Pemrograman/<Kategori>/<Teknologi>/Level N – <Nama>/<Bagian>. <Nama Bagian>/<N. Judul – Subjudul>.md`

- **Nomor file** (`1.`, `2.`, …) = nomor global yang sama dengan nomor di roadmap.
- **Judul file** = sama dengan judul poin roadmap, memakai pemisah `–` (em dash).

---

1. Struktur Baku File Materi (urutan heading wajib)

Ikuti urutan ini **persis**, jangan diacak, jangan dihilangkan:

Urutan	Heading	Isi singkat
1	`# <Judul Materi>`	Judul saja (tanpa nomor di dalam H1)
2	`## Penjelasan`	Apa itu, konsep inti, cara kerja
3	`## Fungsi`	Untuk apa, kenapa ada
4	`## Cara Pengimplementasian / Code`	Contoh kode nyata + cara menjalankan
5	`## Tracing Code (Step-by-Step)`	**Penelusuran eksekusi baris demi baris** (lihat Bagian 3)
6	`## Analogi`	Satu analogi dunia nyata yang relevan
7	`## Dipakai untuk Apa`	Kasus penggunaan nyata
8	`## Kesalahan Umum yang Sering Terjadi`	Jebakan & cara memperbaiki
9	`## Hubungan dengan Materi Sebelumnya`	Kaitan ke materi sebelumnya & sesudahnya
10	`## Soal Latihan & Jawaban`	`### Soal` + `### Jawaban`

> **Nomor di dalam H1 tidak ditulis.** Nomor hanya ada di **nama file** dan di **link roadmap**. Contoh: file `6. Variabel — Wadah Data dengan Tanda Dollar.md` → H1-nya `# Variabel — Wadah Data dengan Tanda Dollar`.

---

2. Aturan Isi Tiap Bagian

`## Penjelasan`

- Mulai dengan definisi 1-2 kalimat yang jelas.
- Lanjutkan penjelasan konsep dan cara kerjanya.
- Gunakan bullet point untuk daftar aturan/fakta penting.
- Jangan langsung masuk kode. Bangun pemahaman dulu.

`## Fungsi`

- Jelaskan **mengapa** materi ini ada dan masalah apa yang dipecahkan.
- Cukup 1-3 paragraf atau beberapa bullet. Sederhana.

`## Cara Pengimplementasian / Code`

- Beri **satu blok kode utama** yang realistis (bukan `foo` / `bar`).
- Pakai konteks yang konsisten di seluruh materi (mis. tema **perpustakaan** untuk roadmap PHP/JavaScript).
- Beri komentar **berbahasa Indonesia** pada baris penting.
- Tutup dengan cara menjalankan/menguji (mis. "Buka `http://localhost/level1/1.php`").
- Sertakan bahasa pada code fence: ` ```php `, ` ```js `, ` ```python `, dll.

`## Analogi` (lihat Bagian 4)

- Satu analogi saja, yang benar-benar terkait.
- Hindari analogi berlapis yang justru menambah kebingungan.

`## Dipakai untuk Apa`

- Bullet list kasus penggunaan nyata (fitur/produk, bukan teori).

`## Kesalahan Umum yang Sering Terjadi`

- 3-6 kesalahan pemula yang paling sering, format: **Nama kesalahan** → penjelasan singkat + akibatnya.
- Sertakan cara menghindarinya bila memungkinkan.

`## Hubungan dengan Materi Sebelumnya`

- Sebutkan nomor & judul materi terkait, jelaskan benang merahnya.
- Contoh: "Setelah tahu cara membuat file `.php` (materi 3-4), sekarang kita simpan data (materi 6)."

`## Soal Latihan & Jawaban`

- `### Soal` : satu soal kontekstual (inline kode bila perlu).
- `### Jawaban` : kode solusi lengkap + comentar. Tidak perlu pembahasan panjang; yang penting bisa dijalankan.

---

3. Bagian Baru: `## Tracing Code (Step-by-Step)`

Ini bagian **wajib tambahan**. Tujuannya: pembaca melihat **apa yang sebenarnya terjadi saat kode berjalan**, tanpa perlu menebak.

Aturan penulisan tracing

1. **Bahasa sederhana, bukan teknis.** Hindari istilah seperti "pointer", "stack frame", "evaluasi ekspresi" tanpa penjelasan. Tulis seperti menjelaskan ke teman.
2. **Jangan pakai analogi di bagian ini.** Analogi sudah ada di bagian `Analogi`. Di sini murni alur eksekusi.
3. **Telusuri baris demi baris (atau blok demi blok)** sesuai urutan eksekusi.
4. **Sebutkan nilai variabel setiap kali berubah.** Contoh: "sekarang `$total` berisi 1".
5. **Untuk loop/rekursi**, gunakan **tabel iterasi** agar jelas.
6. **Akhiri dengan output akhir** yang persis.

Kerangka tracing

````
## Tracing Code (Step-by-Step)

Kita telusuri kode di atas baris demi baris, seolah-olah kita mengikuti
komputer.

```php
$total = 0;
for ($i = 1; $i <= 3; $i++) {
    $total += $i;
}
echo $total;
````

Langkah 1 — baris `$total = 0;`
PHP membuat wadah bernama `$total`, lalu mengisinya dengan angka `0`.

Langkah 2 — baris `for (...)` mulainya putaran
PHP membuat wadah `$i` berisi `1`. Lalu PHP bertanya: "apakah `$i <= 3`?" Karena `1 <= 3` benar, PHP masuk ke dalam blok.

Langkah 3 — baris `$total += $i;`
`+=` artinya "isi lama + isi baru". Jadi `$total = 0 + 1 = 1`. Sekarang `$total` berisi `1`.

Langkah 4 — kembali ke `for`
PHP menambah `$i` jadi `2` (`$i++`). Bertanya lagi: "apakah `2 <= 3`?" Benar. Masuk blok lagi.

Putaran	Nilai `$i`	Dinyatakan `$i <= 3`?	Isi `$total` setelah baris `$total += $i`
1	1	ya	1
2	2	ya	3
3	3	ya	6
—	4	tidak	6 (loop berhenti)
````

Langkah 5 — `echo $total;`
PHP menampilkan isi `$total` terakhir, yaitu `6`.

Output akhir:

```
6
```

```
### Contoh lain yang boleh dipakai
- **Condition:** telusuri cabang mana yang diambil dan mengapa.
- **Function:** beri tahu parameter masuk apa, isi yang dihitung, dan nilai
`return`-nya.
- **Rekursi:** tunjukkan pemanggilan bertingkat dan kapan berhenti.
- **OOP:** tunjukkan kapan `__construct` jalan dan isi tiap property.

---

## 4. Aturan Analogi

Analogi wajib **relevan dan konsisten** sepanjang satu roadmap.
- Untuk roadmap PHP/JavaScript di vault ini, konteksnya **perpustakaan** (rak
buku, pustakawan, formulir anggota).
- Satu analogi per materi. Contoh yang sudah ada: "Variabel = rak buku
berlabel".
- Hindari analogi teknis-ke-teknis yang malah menambah istilah baru.
- Jika konsepnya sulit dianalogikan, boleh 1–2 kalimat singkat; jangan
dipaksakan sampai membingungkan.

---

## 5. Aturan Gaya Bahasa & Kode

- **Bahasa: Indonesia**, santai tapi rapi (boleh "kamu"), tidak kaku, tidak
lebay.
- **Istilah teknis boleh dipakai**, tapi beri arti singkat saat pertama
muncul.
- **Jangan menumpuk jargon** dalam satu kalimat. Satu ide per kalimat.
- **Code comment berbahasa Indonesia.**
- **Konsisten** dengan materi lain: format kode, gaya penamaan variabel contoh
(`$judulBuku`, `$katalog`, dst).
- **Panjang wajar**: 60–150 baris per file. Jangan terlalu pendek (tidak
lengkap) atau terlalu panjang (membosankan).

---
```

````
## 6. Template Lengkap (copy-paste)

```markdown
# <Judul Materi — Subjudul>

## Penjelasan
<definisi + konsep + cara kerja + bullet aturan>

## Fungsi
<untuk apa, kenapa ada>

## Cara Pengimplementasian / Code
```<bahasa>
<kode utama + komentar Indonesia>
````

<Cara menjalankan/menguji: ...>

Tracing Code (Step-by-Step)

<telusuri baris demi baris, sebut nil variabel, tabel iterasi bila perlu, output akhir>

Analogi

<analogi perpustakaan / dunia nyata yang relevan>

Dipakai untuk Apa

- <kasus penggunaan 1>
- <kasus penggunaan 2>

Kesalahan Umum yang Sering Terjadi

- : <penjelasan + akibat>
- : <penjelasan + akibat>

Hubungan dengan Materi Sebelumnya

<materi sebelumnya → materi ini → materi selanjutnya>

Soal Latihan & Jawaban

Soal

Jawaban

<solusi>

---

7. Checklist Kualitas Materi

- [ ] H1 = judul tanpa nomor, sama dengan judul di roadmap.
- [ ] Ada **semua** 10 bagian sesuai urutan.
- [ ] Ada bagian **Tracing Code (Step-by-Step)** dengan bahasa sederhana (tanpa analogi).
- [ ] Kode memakai konteks yang konsisten (perpustakaan) & komentar bahasa Indonesia.
- [ ] Ada minimal 3 kesalahan umum.
- [ ] Ada `### Soal` dan `### Jawaban` yang bisa dijalankan.
- [ ] Bagian "Hubungan dengan Materi Sebelumnya" menyebut nomor/judul materi terkait.
- [ ] Tidak ada istilah teknis yang muncul mendadak tanpa penjelasan.

File terkait:
- `Format Roadmap.md` — induk yang menomori poin & menautkan ke file ini.
- `Format Pembuatan Project.md` — roadmap latihan project.
- `Format Penjelasan Materi Project.md` — format materi untuk project.